



Тест-4-1

Вам не разрешен просмотр этого теста. ✕

|→ **Открыто с:** Friday, 6 March 2026, 22:03 ... **Закрывается:** Saturday, 7 March 2026, 22:03

Разрешено попыток: 1
Ограничение по времени: 10 мин.

Ваши попытки

ПОПЫТКА 1

Состояние	Тест начат	Завершен	Затраченное время
Завершены	Saturday, 7 March 2026, 13:17	Saturday, 7 March 2026, 13:24	7 мин. 24 сек.

Не разрешается

У Вас больше нет попыток

Вопрос 1

Пока нет ответа

Отметить вопрос

Для какого отношения в презентации указано, что оно используется для объектов, которые имеют отношение «имеет (has-a)» друг с другом, где сложный объект называется целым или родительским, а простой — дочерним?

Выберите один ответ:

- ☐ а. Ассоциация.
- ☒ б. Композиция.
- ☐ в. Полиморфизм.
- ☐ г. Наследование.

[Очистить мой выбор](#)

Следующая страница

Вопрос 2

Пока нет ответа

Отметить вопрос

Рассмотрим синтаксис из слайда 8: `class B { A objA; public: B(arg-list) : objA(arg-list1); };`. Что означает часть `: objA(arg-list1)` ?

Выберите один ответ:

- ☐ a. Это объявление дружественной функции.
- ☐ b. Это попытка перегрузить оператор присваивания.
- ☒ c. Это список инициализации, который вызывает конструктор объекта `objA` с параметрами `arg-list1`.
- ☐ d. Это присваивание значения члену `objA` после создания объекта.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 3

Пока нет ответа

Отметить вопрос

Сопоставьте типы отношений с их общепринятыми английскими названиями, данными в начале презентации:

Выберите один ответ:

- ☐ a. Наследование — "has-a", Композиция — "is-a".
- ☐ b. Ассоциация — "part-of", Агрегация — "is-a", Композиция — "has-a".
- ☒ c. Ассоциация — "is-a", Агрегация — "has-a", Композиция — "part-of".
- ☐ d. Ассоциация — "has-a", Агрегация — "part-of", Композиция — "is-a".

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 4

Пока нет ответа

Отметить вопрос

В примере с агрегацией класс `Foo` получает объект `Bar` через конструктор: `Foo(Bar bar) { this.bar = bar; }`. Что произойдёт с объектом `Bar`, когда объект `Foo` будет уничтожен?

Выберите один ответ:

- ☐ a. Это приведёт к утечке памяти.
- ☒ b. Объект `Bar` продолжит существовать, потому что `Foo` только заимствовал его.
- ☐ c. Объект `Bar` также будет уничтожен, так как является частью `Foo`.
- ☐ d. Программа вызовет деструктор `Bar` принудительно.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 5

Пока нет ответа

Отметить вопрос

В примере на слайде 4 создаются два пациента (Patient), которые связаны с одним и тем же доктором (Doctor drHouse). Какое отношение между классами Patient и Doctor демонстрирует этот код?

Выберите один или несколько ответов:

- ☐ а. Наследование.
- ☒ б. Ассоциация (связь, но не владение).
- ☐ в. Композиция.
- ☐ г. Агрегация.

Предыдущая страница

Следующая страница

Вопрос 6

Пока нет ответа

Отметить вопрос

Какое преимущество объектной композиции НЕ упоминалось на слайде 10?

Выберите один ответ:

- ☐ a. Изменение реализации внутреннего класса без привлечения внешних клиентов.
- ☐ b. Чистый дизайн API (легче решить, что включать в API, а что скрыть).
- ☐ c. Повторное использование существующего кода.
- ☒ d. Возможность переопределять поведение родительского класса в дочернем (полиморфизм).

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 7

Пока нет ответа

Отметить вопрос

Какое из утверждений НЕ является обязательным свойством композиции (отношения "часть-целое") согласно слайду 5?

Выберите один ответ:

- ☒ а. Часть может принадлежать нескольким объектам одновременно.
- ☐ б. Часть (член) является частью объекта (класса).
- ☐ в. Существование части управляется объектом.
- ☐ г. Часть создается при создании объекта и уничтожается при его уничтожении.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 8

Пока нет ответа

Отметить вопрос

Какой фрагмент кода иллюстрирует отношение **композиции**?

Выберите один ответ:

- ☒ a. `class Point { int x; int y; }; class Location { Point Source; Point Destination; };`
- ☐ b. `class Foo { private Bar bar; Foo(Bar bar) { this.bar = bar; } };`
- ☐ c. `class A {}; class B : public A {};`
- ☐ d. `class Doctor { ... }; class Patient { Doctor* myDoctor; Patient(Doctor* doc) : myDoctor(doc) {} };`

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 9

Пока нет ответа

Отметить вопрос

В примере с классами `Two` и `One` (слайд 7), для доступа к члену `num` класса `One` используется выражение `T.o.set(100)`. Почему нельзя написать просто `T.set(100)`?

Выберите один ответ:

- ☐ a. Потому что класс `Two` не имеет дружественного доступа к классу `One`.
- ☐ b. Потому что метод `set()` объявлен как `private` в классе `Two`.
- ☒ c. Потому что `set()` — это метод класса `One`, а не класса `Two`. Класс `Two` содержит объект класса `One`, но не наследует его интерфейс.
- ☐ d. Потому что компилятор не может разрешить перегрузку функции.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Следующая страница](#)

Вопрос 10

Пока нет ответа

Отметить вопрос

Какое определение наиболее точно описывает отношение **ассоциации** между классами?

Выберите один ответ:

- ☐ a. Объект заимствует другой объект, и при уничтожении владельца заимствованный объект продолжает жить.
- ☐ b. Объект является владельцем другого объекта и управляет его временем жизни.
- ☒ c. Один объект связан с другим, но они не являются частями друг друга, и их жизненные циклы никак не связаны.
- ☐ d. Один класс является частным случаем (подтипом) другого класса.

[Очистить мой выбор](#)

[Предыдущая страница](#)

[Закончить попытку...](#)